PYTHON FOUNDATION CHALLENGE

Week 4 — Functional Thinking + Revision (Weeks 1–3)

Questions 1–10

Objective

Introduce functional programming as a natural next step after three months of Python fundamentals — pure functions, map, filter, comprehensions, higher-order functions, and a first real taste of recursion. This challenge is a gauge: it tells you (and them) where they are, not where they should already be.

A Word Before You Start

You've been learning Python for about three months. You can write loops, use dictionaries, build functions, and solve problems. This week introduces a new style — functional programming — and the questions below are designed to stretch you, not to confirm what you already know.

- Some questions will feel familiar — those are revision with a twist.
- Some will feel new — those are the gauge. Struggling is the point.
- Bonus questions are optional. Do not attempt them until the required ones are done.
- If you get stuck for more than 20 minutes on one question, move on, come back later, or write down what concept is blocking you. That note is more valuable than the answer.

What "functional" means here (short version):

- A pure function takes inputs and returns an output. Same input → same output. It does not print, does not change variables outside itself, and does not depend on anything except its arguments.
- print() belongs outside your functions, not inside them.
- Prefer map, filter, and comprehensions over manual loops where they make the code clearer. Loops are still allowed unless a question says otherwise.
- Recursion is a function that calls itself. It's not always better than a loop — but it's a tool you need to know.

Instructions

- Solve the problems independently. Do not look at previous solutions.
- Focus on producing a working solution first; clean up the code afterward if time allows.
- No external libraries unless a question explicitly permits them.
- If you get stuck, identify the Python concept causing the problem rather than immediately searching for a solution.
- Functional theme: prefer pure functions, avoid mutating shared state, use map/filter/comprehensions where they help. Loops are allowed unless stated otherwise.
- Bonus questions are optional stretch goals.

Pure vs. Impure — See the Difference

❌ **Impure version** — prints inside the function, does two jobs at once:

Examples

```
def is_even(n):
```

```
        if n % 2 == 0:
            print("Even")
        else:
            print("Odd")


    is_even(4)   # prints "Even"
    is_even(4)   # prints "Even" again — but you can't reuse the result
```

✅ **Pure version — returns a value, prints nothing:**

Examples

```
def is_even(n):
    return n % 2 == 0

result = is_even(4)   # result is True
print("Even" if result else "Odd")   # printing happens OUTSIDE
```

Week 4 at a Glance

The progression:

```
Pure functions  →  map / filter  →  higher-order functions  →  recursion  →  integration

  (Q1, Q3, Q4, Q5)   (Q2)          (Q6, Q7)                  (Q8)         (Q10)
```

Part A — Revision (Higher Bar)

These revisit Weeks 1–3. The logic is the same — the style is what's new. Expect to spend more time than last week on the same problems.

1. Palindrome (Revision — Clean Logic)

Suggested time: 20–25 minutes

Problem

Write a program that determines whether a full sentence reads the same forwards and backwards, ignoring mixed casing, spaces, and punctuation.

Examples

```
Input: Was it a car or a cat I saw? → Output: Palindrome

Input: Hello, World! → Output: Not a palindrome
```

Constraints

- Must use a strict two-pointer approach (left and right indices moving inward).
- Do not create reversed strings, substrings, or use [::-1] / .reverse().
- Manually skip spaces and punctuation during pointer comparison.

- Split the logic into two pure functions: is_letter(ch) -> bool and is_palindrome(text) -> bool.
- The print() call must live outside both functions.
- Do not import libraries.

🟢 Bonus

Rewrite is_palindrome as a recursive function is_palindrome(text, left, right) with no while loop.

What this gauges

Can you separate logic from output? Can you write a function that returns a value instead of printing it?

2. Student Grades (Revision — now with map / filter)

Suggested time: 25–30 minutes

Problem

Given a list of dictionaries representing students and their scores, calculate each student's average and assign a letter grade.

Examples

```
students = [
    {"name": "Sam", "scores": [80, 90]},
    {"name": "David", "scores": [55, 60]}
]

Sam → Average: 85.00 → Grade: A
David → Average: 57.50 → Grade: C
```

Grading System

70–100 → A

60–69 → B

50–59 → C

45–49 → D

40–44 → E

0–39 → F

Constraints

- Calculate the average manually (no sum(), no statistics.mean()). A for loop inside a helper is fine.
- The grade logic must be a pure function grade_for(avg) -> str — no printing inside.
- Use map at least once to produce a list of (name, average, grade) tuples.
- Use filter at least once to produce a list of students who passed (grade A–C).
- Track and print the highest and lowest performing students manually.
- No max() or min() on the averages — use variables and if statements.
- Use the grading system from Week 1.
- Do not import libraries.

🟢 Bonus

Replace the map call with a list comprehension. Which version do you find clearer? Write one sentence explaining why.

What this gauges

Can you use map and filter as replacements for a loop, not just as decorations? Can you write a small pure function (grade_for) and reuse it?

3. Word Frequency (Revision — Pure Core)

Suggested time: 20–25 minutes

Problem

Given a paragraph of text, count how many times each word appears and store it in a dictionary. Then find and print the single most frequent word.

Examples

```
Input: "Python is fun, and Python is powerful!"
Output: Most frequent word: python (2)
```

Constraints

- Treat uppercase and lowercase as the same.
- Strip basic punctuation (commas, periods) manually — no re, no string.punctuation.
- Create two pure functions: count_words(words: list[str]) -> dict[str, int] and most_frequent(freq: dict[str, int]) -> str.
- count_words builds a fresh dict and mutates nothing from outside.
- most_frequent returns the most frequent word; a for loop is allowed.
- No max() on the dictionary. No sorting. No collections.Counter.
- Do not import libraries.

🟢 Bonus

Rewrite most_frequent using functools.reduce (from functools import reduce allowed only for this bonus).

What this gauges

Can you build a dictionary inside a function without touching anything outside it? This is the heart of no side effects.

4. Two Sum (Revision — Pure Function)

Suggested time: 20–25 minutes

Problem

Given a list of numbers and a target number, find the first valid pair whose sum equals the target. Return their indices.

Examples

```
Numbers: [2, 7, 11, 15] | Target: 9 → Output: [0, 1]
```

Constraints

- Return indices, not the numbers.
- O(n) time using a dictionary of seen numbers.
- Nested loops are forbidden.
- The two numbers must come from different positions.
- Use a pure function two_sum(nums, target) -> list[int] returning [] if no pair exists. No printing inside.
- A for loop inside two_sum is fine — this is about purity, not banning loops.
- Do not import libraries.

🟢 Bonus

Rewrite the "seen so far" tracking using functools.reduce, carrying the dict as the accumulator.

What this gauges

Can you keep the O(n) hashmap insight from Week 3 and wrap it in a clean, testable function?

5. Second Largest (Revision — Single Pass, Pure)

Suggested time: 20–25 minutes

Problem

Given a list of numbers, find the second-largest unique number.

Examples

```
Input: [10, 5, 8, 20, 15] → Output: 15

Input: [4, 9, 2, 9, 7] → Output: 7
```

Constraints

- No sort(), sorted(), max(), or set() conversion.
- Track largest and second_largest manually in a single pass.
- Duplicates must not affect the result.
- The main second_largest(nums) -> int uses a for loop and calls step each iteration.

🟢 Bonus

Rewrite as a single functools.reduce call where the accumulator is (largest, second_largest).

What this gauges

Can you pull the update rule out of a loop and make it a standalone function? This is the mental move that unlocks reduce later.

6. Caesar Cipher (Revision — Higher-Order Function)

Suggested time: 25–30 minutes

Problem

Encrypt or decrypt a message by shifting letters. Shift of 1: 'a' → 'b', 'z' → 'a'.

Examples

```
hello | Shift: 1 | Mode: encrypt → ifmmp

xyz | Shift: 2 | Mode: encrypt → zab

hello, world! | Shift: 3 | Mode: encrypt → khoor, zruog!

ifmmp | Shift: 1 | Mode: decrypt → hello
```

Constraints

- Handle wrap-around correctly.
- Preserve spaces, digits, and punctuation.
- The program asks the user whether to encrypt or decrypt.
- Build a higher-order function make_shifter(shift) -> function. The returned function takes a single character and returns the shifted character.
- Use map to apply the shifter across the message.

- The per-character shift must be a pure function — no printing, no globals.
- Encrypt and decrypt use the same make_shifter (negative shift for decryption).
- No external libraries.

🟢 Bonus

Make make_shifter handle both uppercase and lowercase letters without writing the alphabet twice.

What this gauges

Can you write a function that returns another function? This is your first real closure — and the foundation of decorators later.

Part B — New Questions: Functional Progression

New problems. Loops are allowed unless the question says otherwise. The point is to think in functions, not to ban every for.

7. Compose and Pipe

Suggested time: 20–25 minutes

Problem

Write two higher-order functions: compose(f, g) → returns a function computing f(g(x)); pipe(*funcs) → returns a function applying functions left-to-right: pipe(f, g, h)(x) == h(g(f(x))). Then build a small text pipeline: strip whitespace → lowercase → remove vowels → reverse.

Examples

```
add_one = lambda x: x + 1
double  = lambda x: x * 2
compose(add_one, double)(5)   # 11
pipe(add_one, double)(5)      # 12

pipeline = pipe(strip, lower, remove_vowels, reverse)
pipeline("  Hello World  ")
```

Constraints

- pipe must accept any number of functions (use *funcs).
- Each helper (strip, lower, remove_vowels, reverse) must be a pure function taking a string and returning a string.
- reverse must not use [::-1] or .reverse() — write a manual loop.
- Loops are allowed inside the helpers. The functional part is compose and pipe themselves.
- No external libraries.

🟢 Bonus

Rewrite reverse as a recursive function with no loop.

What this gauges

Can you treat functions as values — pass them around, return them, chain them? This is the conceptual leap from functions to functional programming.

8. Recursive Flatten and Group

Suggested time: 25–30 minutes

Problem

Two parts:
1. flatten(nested) — take a list that may contain other lists (any depth) and return a flat list of all non-list elements.
2. group_by(items, key_fn) — return a dict mapping key_fn(item) → list of items.
Then combine: given a nested list of words, flatten it and group by first letter.

Examples

```
flatten([1, [2, [3, [4]], 5]])  # [1, 2, 3, 4, 5]
group_by(["apple", "avocado", "banana"], lambda w: w[0])
# {'a': ['apple', 'avocado'], 'b': ['banana']}

combined([["apple", "avocado"], ["banana"], [["cherry"]]])
# {'a': ['apple', 'avocado'], 'b': ['banana'], 'c': ['cherry']}
```

Constraints

- flatten must be recursive — no for/while inside it.
- group_by may use a for loop, but must be a pure function: build a fresh dict inside and mutate nothing passed in.
- key_fn must be a parameter (higher-order function), not hardcoded.
- No external libraries.

🟢 Bonus

Rewrite group_by as a recursive function with no loop.

What this gauges

Can you see a self-similar structure (a list inside a list inside a list) and write a function that solves the small case and trusts itself for the rest? This is the recursion aha.

9. Memoized Fibonacci (Bonus / Optional)

Suggested time: 25–30 minutes

Note: Decorators are normally a later topic. This question is intentionally bonus — attempt it only after Q1–Q8, and only if you're curious.

Problem

Write a decorator memoize that caches a function's results. Apply it to a recursive Fibonacci function and show that fib(35) runs fast.

Examples

```
@memoize
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

fib(35)   # 9227465, almost instantly
```

Constraints

- memoize must be a decorator working on any function with hashable args (use *args).
- The cache must live in the closure of the decorator — not as a global.
- The decorated function must return the correct result every time.

- No external libraries (functools.wraps is optional).

🟢 Bonus (of the bonus)

Apply memoize to a second function — e.g., a recursive count_ways(n) for climbing stairs with 1 or 2 steps — to prove it generalises.

What this gauges

Can you combine closures (from Q6) with recursion (from Q8) to solve a problem that's otherwise intractable? This is the summit of the week.

10. Transaction Analyzer (Integrated Functional Challenge(Recommended))

Suggested time: 30–40 minutes

Problem

Build a program that analyzes a list of financial transactions. Your program should calculate total income, total expenses, balance, largest expense, number of expenses, and overall financial status.

Starting Data

```
transactions = [
    {"name": "Laptop", "amount": 250000, "type": "expense"},
    {"name": "Salary", "amount": 500000, "type": "income"},
    {"name": "Internet", "amount": 30000, "type": "expense"},
    {"name": "Freelance", "amount": 150000, "type": "income"},
    {"name": "Food", "amount": 45000, "type": "expense"}
]
```

Expected Output

```
Total income: 650000

Total expenses: 325000

Balance: 325000

Largest expense: Laptop (250000)

Number of expenses: 3

Status: Positive balance
```

Constraints

- Do not use external libraries.
- Do not modify the original transactions list.
- Break the solution into small, focused functions.
- Your analysis functions should return values rather than print them.
- Keep printing/output outside the analysis functions.
- Create: get_income, get_expenses, calculate_total, find_largest, get_balance, and get_status.
- get_income returns only income transactions.
- get_expenses returns only expense transactions.
- Use filter at least once.
- Use map at least once to extract amounts.
- calculate_total must calculate the total manually; do not use sum().
- find_largest must find the largest expense manually; do not use max() or sorted().
- The original list must remain unchanged.

- Handle no transactions, no expenses, and a balance of exactly 0.

Edge Cases

```
[]
→ Income: 0, Expenses: 0, Balance: 0, Status: Break-even

Only salary:
[{"name": "Salary", "amount": 500000, "type": "income"}]
→ Income: 500000, Expenses: 0, Balance: 500000
→ Status: Positive balance
→ Largest expense: None
→ Expense count: 0
```

🟢 Bonus

Create filter_transactions(transactions, condition), then use it with a lambda for income transactions.

🔴 Hard Bonus

```
analyze(transactions)

# returns:
{
    "income": 650000,
    "expenses": 325000,
    "balance": 325000,
    "largest_expense": "Laptop",
    "expense_count": 3,
    "status": "Positive balance"
}
```

What this gauges

- Breaking a larger problem into smaller functions
- Pure functions
- map and filter in an actual problem
- Lists of dictionaries
- Manual tracking
- Edge cases
- Function reuse
- Separating processing from output
- Program structure rather than isolated algorithms

Progression

Familiar problems, rewritten as pure functions → functions that take and return other functions → recursive thinking on nested data → an optional first look at decorators and closures → an integrated problem where you must decide how to combine the tools yourself.

The through-line is purity and clarity: every constraint pushes you to write functions that do one thing, take clear inputs, and return clear outputs.

Time Guide

Section	Questions	Suggested Time

Revision (higher bar)	1–6	2 hrs 15 min – 2 hrs 45 min
New progression (required)	7–8	45 – 55 min
New progression (bonus)	9	25 – 30 min
Integrated challenge	10	30 – 40 min
Overall (required)	1–8 + 10	About 3 hrs 30 min – 4 hrs 20 min
Overall (with bonus)	1–10	About 4 hrs – 4 hrs 50 min

If you're short on time, the core five from the functional progression remain Q2, Q3, Q6, Q7, Q8. Q10 is the best integration/gauge question and can be used as the final challenge.

Self-Assessment Guide

Question	Concept being tested	What it tells you
1. Palindrome	Pure functions, separating logic from output	Can you return values instead of printing inside functions?
2. Grades	map, filter, reusable pure functions	Can you use map/filter meaningfully?
3. Word Frequency	Building a dict inside a function, no mutation	Do you understand side effects?
4. Two Sum	Hashmap + purity	Can you preserve the O(n) insight in a clean function?
5. Second Largest	Extracting an update rule	Can you pull loop logic into a reusable function?
6. Caesar Cipher	Higher-order functions, closures	Can you write a function that returns a function?
7. Compose / Pipe	Functions as values	Can you chain functions?
8. Flatten	Recursion on nested data	Can you see and solve self-similar structure?
9. Memoize	Closures + recursion + decorators	Can you combine advanced concepts?
10. Transaction Analyzer	Integration and program structure	Can you choose and combine the tools without being told exactly where each one belongs?

How to Read Your Results

- 5–6 required questions solved cleanly → You're on track. Functional thinking is starting to click.
- 3–4 required questions solved → You have the fundamentals; the new style needs more practice. That's expected at this stage.
- 1–2 required questions solved → The functional mindset hasn't landed yet. That's useful information — not a failure.
- Q9 solved → You're ready to go deeper into functional Python.
- Q10 solved cleanly → You can integrate multiple concepts into a structured program rather than solving only isolated exercises.

A Note on "Gauging"

This challenge is a gauge, not an exam. The point is to find out which functional ideas feel natural, which ones need another pass, and where your existing Week 1–3 knowledge is solid vs. shaky.

Struggling is data, not failure. If you finish half of this and can name exactly which concept blocked you on the rest, you've learned more than someone who copied answers to all ten.

Write down your blockers. Bring them to the next session. That list is the real deliverable.

Notes for the Instructor

Design choice	Reason
Loops allowed except where explicitly restricted	Removes unnecessary 'I don't know where to start' cliffs while preserving the functional focus.
reduce is bonus-only	Keeps the required path achievable for learners at this stage.
Q9 is clearly bonus	Decorators are an advanced stretch; students can attempt them without making them the baseline.
Recursion required in Q8	Flattening is the canonical recursion challenge.
Pure-function constraints throughout	This is the main functional habit being measured.
Q6 introduces closures	A natural bridge from functions to decorators.
Q10 integrates the week	Students must make architectural decisions instead of being told which functional tool to use at every step.

End of Week 4 Challenge